

TherapistTrainer Tutorial Video

End-to-end procedure: from the ScreenFlow project file to a single .mp4 carrying toggleable English / Swedish / Spanish / German subtitle tracks

This document is the reference procedure for regenerating subtitles for the ThTrainer tutorial video (v2.0 / v2.1), and for adding further languages later. It covers four stages, run in order:

- **1. Extract clip timing** from the ScreenFlow project's Core Data store (`extract_mp3_clips.py`).
- **2. Build the English subtitles** by force-aligning the known narration script against each section's real audio (`create_srt_file.py`, `WhisperX`).
- **3. Build translated subtitles** for any additional language, once a human translation exists (`create_translated_srt.py`).
- **4. Attach every subtitle track to the video** in one pass, as soft/toggleable tracks (`attach_srt_to_video.py`).

Repository layout

```
src
  ■■■ aligning
  ■   ■■■ __init__.py
  ■   ■■■ audio_files_and_timings.txt    ← output of Stage 1, input to Stages 2 & 3
  ■   ■■■ create_srt_file.py             ← Stage 2 (English, forced alignment)
  ■   ■■■ create_translated_srt.py       ← Stage 3 (translated languages)
  ■■■ attaching
  ■   ■■■ __init__.py
  ■   ■■■ attach_srt_to_video.py         ← Stage 4
  ■■■ screen_flow_extraction
  ■   ■■■ __init__.py
  ■   ■■■ extract_mp3_clips.py           ← Stage 1
  ■   ■■■ ScreenFlowDocument.dat        ← copied out of the .screenflow project bundle
```

Convention used throughout this document: `$PROJ_ROOT` is the tutorial video project folder, e.g. `/Users/paepcke/Project/TherapistTrainer/VideoIntro/ThTrainerV2_0`.

Stage 1 — Extract clip timing from the ScreenFlow project

ScreenFlow's project file (`ScreenFlowDocument.dat`, inside the `.screenflow` bundle) is a Core Data "Binary Store" (an `NSKeyedArchiver`-serialized object graph). Each recorded clip is a generic row: an entity name, a primary key, and an *ordered array* of attribute values with no field names attached. To know which array slot is `startTime` vs `duration`, the script reads that same ordering out of ScreenFlow's own compiled Core Data model (a `.mom` file inside the `ScreenFlow.app` bundle) — matched to the document by a checksum ScreenFlow itself records, so the right model version is found automatically.

- No third-party packages needed — uses only `plistlib` (standard library).
- Requires ScreenFlow's compiled model folder. If it isn't at the default `/Applications/ScreenFlow.app/Contents/Resources/Scarlett.momd`, adjust `MOMD_DIR` at the top of the script — find it with `find /Applications/ScreenFlow*.app -iname "*.momd"`.
- Copy the project's `ScreenFlowDocument.dat` next to the script before running (the script looks for it in its own directory).

Run it

```
cd src/screen_flow_extraction
python3 extract_mp3_clips.py > ../aligning/audio_files_and_timings.txt
```

The script prints one row per on-timeline .mp3 clip: absolute start time, on-timeline duration, clip type, and the source file path — sorted by start time. Redirecting stdout into `aligning/audio_files_and_timings.txt` puts it exactly where Stages 2 and 3 expect it by default.

Sample output row: `00:15.60 | 4.63s | MediaClip | .../section1a_{5EFAB93A-...}.mp3`

Stage 2 — Build the English subtitles (forced alignment)

`create_srt_file.py` already knows the exact narration text for each section (`script_v2_1.txt`) and the exact real audio for it (via the clip table from Stage 1). Rather than transcribing the audio blind, it uses WhisperX's forced-*alignment* step only — `whisperx.align()` — to find exactly when each known word is spoken. Each section is aligned independently against its own .mp3, so no section's timing can drift into a neighboring section, and the resulting word timestamps are then offset into the master timeline.

Script and audio-script blocks are matched by **position**, not by their text labels — the labels are known to be inconsistent (“session” vs “section”, non-sequential/duplicate numbering), so row N of the clip table is always paired with block N of the script.

Bug this script works around: WhisperX's `align()` expects roughly one sentence per input segment — the shape its own ASR step produces. Feeding it one segment containing an entire multi-sentence section silently aligns *only the first sentence* and drops every sentence after it — no error, no warning. `create_srt_file.py` splits each section into per-sentence segments (seeded with a word-count-proportional time estimate) before calling `align()`, which alignment then refines against the real audio. Always spot-check a fresh run's .srt against the script if this logic is ever changed.

Run it

```
python3 src/aligning/create_srt_file.py \
  --audio-script $PROJ_ROOT/script_v2_1.txt \
  --out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1.srt \
  --max-chars 40
```

Key options:

- `--clip-table` — defaults to `audio_files_and_timings.txt` next to the script (Stage 1's output).
- `--audio-script` (required) — the narration text, in section blocks.
- `--max-chars` — max characters per subtitle cue (default 32; 40 used above).
- `--language` — WhisperX alignment-model language code (default en).
- `--out-boundaries` — optional; defaults to `text_timing_<language>.json` in the current directory, and auto-suffixes (`_1`, `_2`, ...) instead of overwriting an existing file.

Wrote 124 cues to `$PROJ_ROOT/thtrainer_2_0_script_2_1.srt`
Wrote 26 section boundaries to `text_timing_2.json`

Output files:

SRT: `$PROJ_ROOT/thtrainer_2_0_script_2_1.srt`
Boundaries: `text_timing_2.json`

The boundaries JSON records each section's absolute [start, end] in the final video — the anchor points a future dubbing phase (synthesized foreign-language *audio*, not yet undertaken) will need. It depends only on the clip-timing table, never on subtitle language, so it only needs regenerating here in Stage 2 — see the note in Stage 3.

Stage 3 — Build translated subtitles

There is no foreign-language *audio* for these tracks — the video's narration audio stays in English regardless of which subtitle track a viewer picks. This stage has two parts: a human translation step, then a timing step.

3a. Translate the script (manual / human step)

Neither WhisperX nor ElevenLabs is the right tool here — this is plain translation. Produce a text file in **exactly the same block format** as `script_v2_1.txt`:

- The same `sectionN` label lines, copied **verbatim** and in the same order, so the tooling can zip this file against the clip-timing table by position, exactly as Stage 2 does.
- Quoted UI element names (button/box labels like “Client History”, “Send”, “Session Critique”) left in English, since the on-screen application itself is not localized.
- A native-speaking clinician should review the draft before it ships, especially the ISTDP-specific terminology.

3b. Time the cues

`create_translated_srt.py` takes each section's real, already-known [start, start+duration] window from the same clip-timing table Stage 1 produced, so section boundaries are exact. Within a section, though, there is no foreign speech to time against — so the script splits the translated text into subtitle-sized cues and distributes the section's real duration across them **proportional to each cue's character count**. This is a solid starting point, not a final cut — if a cue looks oddly paced on playback, nudge it by hand (e.g. in Aegisub).

Run it (once per language)

```
python3 src/aligning/create_translated_srt.py \
  --translated-script $PROJ_ROOT/script_v2_1_swedish.txt \
  --out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_swedish.srt \
  --max-chars 40

python3 src/aligning/create_translated_srt.py \
  --translated-script $PROJ_ROOT/script_v2_1_spanish.txt \
  --out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_spanish.srt \
  --max-chars 40

python3 src/aligning/create_translated_srt.py \
  --translated-script $PROJ_ROOT/script_v2_1_german.txt \
  --out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_german.srt \
  --max-chars 40
```

This script deliberately writes no boundaries JSON. Section boundaries come from `audio_files_and_timings.txt` alone — they don't depend on subtitle language or text — so Stage 2's `create_srt_file.py` writing one, once, is the single source of truth. A per-language copy here would just be an identical duplicate under a different name.

Stage 4 — Attach every subtitle track to the video

`attach_srt_to_video.py` layers one or more `.srt` files onto the `.mp4` as soft, toggleable subtitle tracks — not burned into the picture. Video and audio are stream-copied (`-c copy`); only the subtitle text is transcoded, into `mov_text`, the one subtitle codec MP4 containers support.

Always attach every language in a single run, from the ORIGINAL unsubtitled video — not by pointing --video at a previously subtitled output to layer one more language on top. Two real, silent ffmpeg bugs were found and confirmed by direct testing:

1. Daisy-chaining wipes track titles. Attach English, re-run to add Swedish onto that output, re-run again to add Spanish onto that — by the third run, English's and Swedish's *titles* are both gone (players fall back to a generic “Track N”), even though their *language* tags survive. Confirmed this happens after just **one** remux hop, not only by the third. Language survives because it lives in the MP4 track header; title lives in a separate metadata atom that ffmpeg's mov demuxer does not reliably re-expose on a second remux. Attaching every track in one ffmpeg invocation has only one generation, so the bug never triggers.

2. A 2-letter language code fails silently. MP4 requires the 3-letter ISO 639-2 code (spa), not the 2-letter ISO 639-1 code (es). The wrong length doesn't error — ffmpeg just drops the language tag with no message. The script now checks for a 3-letter alphabetic code up front and refuses to run otherwise.

Run it

```
python3 src/attaching/attach_srt_to_video.py \  
  --video $PROJ_ROOT/FinalProduct/therapistTrainerTutorial_2_1.mp4 \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_english.srt --language eng --label English \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_swedish.srt --language swe --label Swedish \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_spanish.srt --language spa --label Spanish \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_german.srt --language deu --label Deutsch \  
  --out $PROJ_ROOT/therapistTrainerTutorial_2_1_subtitled.mp4
```

--srt / --language are each repeated once per track, in matching order; --label is optional but, if given, must be given once per track too. To add a language later, re-run the full command above (from the original video) with the new track added — don't chain onto the already-subtitled output.

Reference: gotchas and why the scripts are shaped this way

Symptom	Cause	Fix
WhisperX .srt silently missing every sentence after a section's first	align() expects ~one sentence per input segment; a whole multi-sentence section as one segment truncates silently	Split each section into per-sentence seed segments before calling align() (create_srt_file.py)
No way to time translated cues precisely within a section	No foreign-language audio exists to force-align against	Distribute each section's real, known duration across cues proportional to character count (create_translated_srt.py)
VLC subtitle menu shows generic "Track N" instead of the language name after adding one more language	Daisy-chained ffmpeg remuxing drops previously-set track TITLE metadata (language tag survives)	Attach every language in one single ffmpeg invocation, from the original subtitled video
A subtitle track ends up with no language tag at all, no error shown	2-letter ISO 639-1 code ("es") given where MP4 requires the 3-letter ISO 639-2 code ("spa")	attach_srt_to_video.py validates for a 3-letter code up front and refuses to run otherwise
Multiple duplicate text_timing_N.json files accumulating	Boundaries only depend on the clip-timing table, never on subtitle language	Only create_srt_file.py (Stage 2) writes this file, once per project; create_translated_srt.py writes none

Appendix: full command reference

PROJ_ROOT=/Users/paepcke/Project/TherapistTrainer/VideoIntro/ThTrainerV2_0

Stage 1 — extract clip timing

```
cd src/screen_flow_extraction
python3 extract_mp3_clips.py > ../aligning/audio_files_and_timings.txt
```

Stage 2 — English

```
src/aligning/create_srt_file.py \
--audio-script $PROJ_ROOT/script_v2_1.txt \
--out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1.srt \
--max-chars 40
```

Stage 3 — Swedish

```
src/aligning/create_translated_srt.py \
--translated-script $PROJ_ROOT/script_v2_1_swedish.txt \
--out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_swedish.srt \
--max-chars 40
```

Stage 3 — Spanish

```
src/aligning/create_translated_srt.py \
--translated-script $PROJ_ROOT/script_v2_1_spanish.txt \
--out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_spanish.srt \
--max-chars 40
```

Stage 3 — German

```
src/aligning/create_translated_srt.py \  
  --translated-script $PROJ_ROOT/script_v2_1_german.txt \  
  --out-srt $PROJ_ROOT/thtrainer_2_0_script_2_1_german.srt \  
  --max-chars 40
```

Stage 4 — attach all subtitles at once

```
src/attaching/attach_srt_to_video.py \  
  --video $PROJ_ROOT/FinalProduct/therapistTrainerTutorial_2_1.mp4 \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_english.srt --language eng --label English \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_swedish.srt --language swe --label Swedish \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_spanish.srt --language spa --label Spanish \  
  --srt $PROJ_ROOT/thtrainer_2_0_script_2_1_german.srt --language deu --label Deutsch \  
  --out $PROJ_ROOT/therapistTrainerTutorial_2_1_subtitled.mp4
```